AZURE CLOUD FUNDAMENTALS
AND
DATA PIPELINE IMPLEMENTATION USING AZURE DATA FACTORY

Week 4 Assignment Report

Assignment	**Week 4**
Project	**Azure Data Factory Pipeline**
Submitted By	**Eklavya (MMDU)**
Organization	**Celebal Technologies**
Technology	**Microsoft Azure**

1. Introduction

Cloud computing has become one of the most important technologies in modern software development and data engineering. Microsoft Azure provides a wide range of cloud services that help organizations store, process, and analyze data efficiently. Azure Data Factory (ADF) is a cloud-based data integration service that enables users to create, schedule, and manage data pipelines for moving and transforming data between different sources and destinations.

In this assignment, Azure cloud services were explored by creating a Resource Group, Storage Account, Blob Containers, and Azure Data Factory. An end-to-end data pipeline was developed to read a CSV file from Azure Blob Storage, validate its metadata, and copy the file to another location within the storage account. The project also involved monitoring pipeline execution and configuring access permissions using Azure IAM roles.

---

2. Objective

The primary objective of this assignment was to understand Azure cloud concepts and implement a complete data pipeline using Azure Storage Account and Azure Data Factory.

The assignment focused on learning how to create Azure resources, configure storage services, establish connectivity through Linked Services, create datasets, validate metadata, perform data movement operations, and monitor pipeline execution. Additionally, the assignment aimed to provide practical experience in managing access permissions through Azure Identity and Access Management (IAM).

---

3. Technologies and Services Used

The following Azure services and technologies were used during the implementation of this project:

- Microsoft Azure Portal
- Azure Resource Group
- Azure Storage Account
- Azure Blob Storage
- Azure Data Factory (ADF)
- Azure Identity and Access Management (IAM)
- CSV Dataset (Sample-Superstore.csv)

---

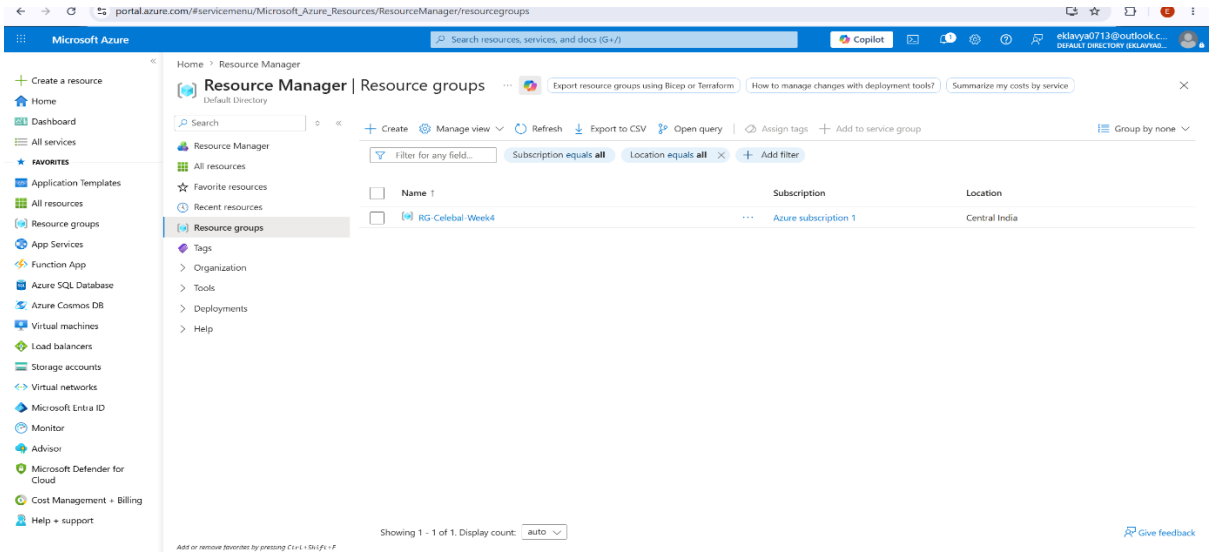
4. Resource Group Creation

The first step of the implementation was to create a Resource Group in Microsoft Azure. A Resource Group named **RG-Celebal-Week4** was created in the Central India region. The

Resource Group acts as a logical container that helps organize and manage all Azure resources used in the project.

Creating a Resource Group makes it easier to monitor, secure, and manage related resources from a single location. All project resources, including the Storage Account and Azure Data Factory instance, were created inside this Resource Group.

Screenshot_01_Resource_Group_Created



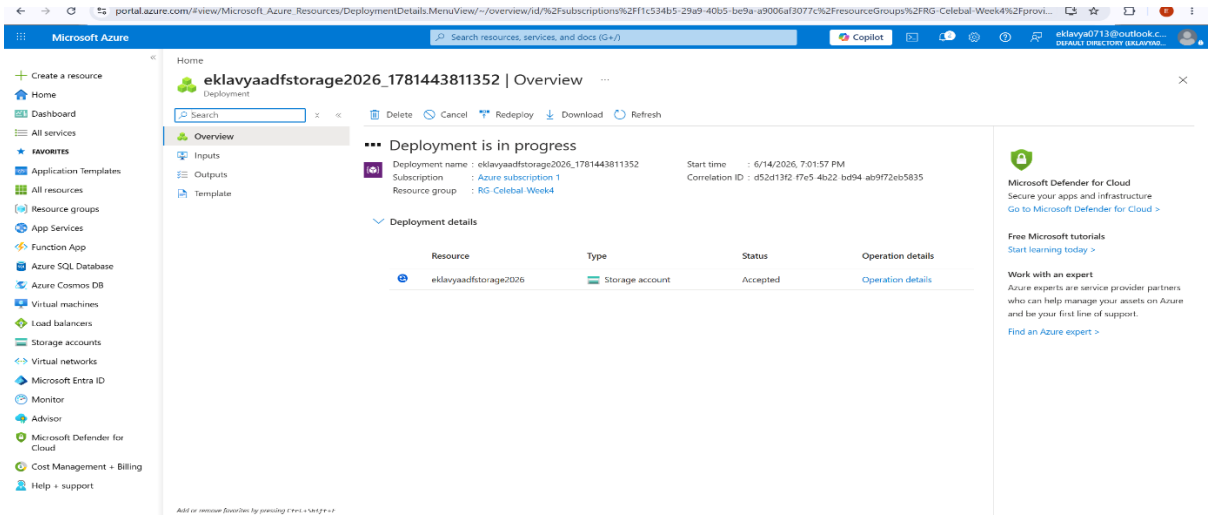
---

5. Storage Account Creation

After creating the Resource Group, a Storage Account named **eklavyaadfstorage2026** was created. The Storage Account serves as the primary cloud storage service used to store input and output files for the data pipeline.

The Storage Account was configured using standard performance settings and locally redundant storage (LRS), which ensures data availability and durability within the selected Azure region.

Screenshot_02_Storage_Account_Created



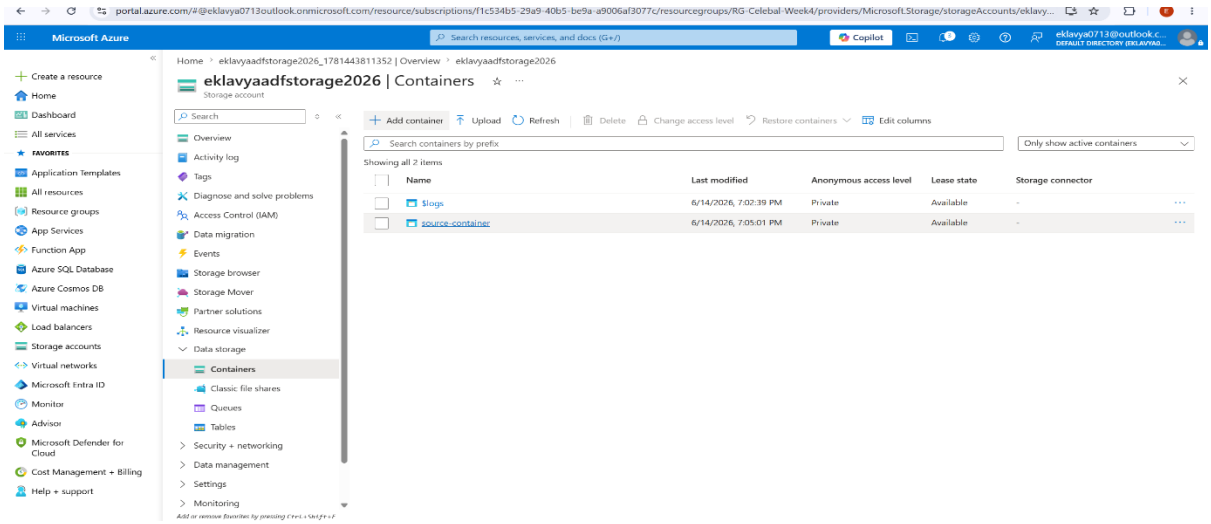
6. Blob Container Creation

Inside the Storage Account, two Blob Containers were created to store the source and destination files.

The first container, named **source-container**, was used to store the original CSV file. The second container, named **destination-container**, was created to store the output file generated after successful pipeline execution.

Using separate containers for source and destination data helps maintain proper organization and clearly demonstrates the data movement process.

Screenshot_03_Blob_Containers_Created

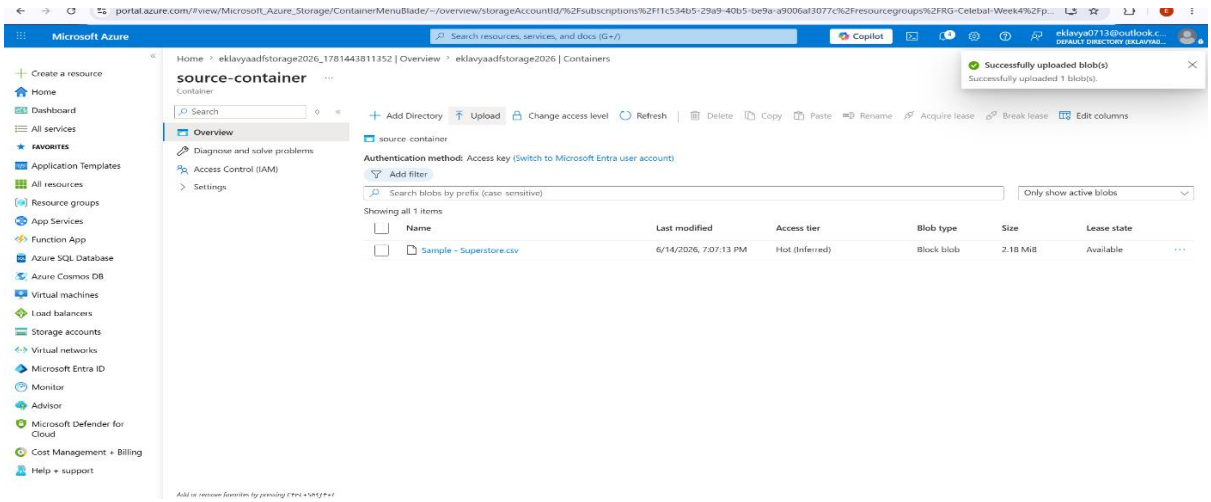


7. Uploading Source Dataset

A CSV dataset named **Sample-Superstore.csv** was uploaded to the source-container. This file served as the input dataset for the Azure Data Factory pipeline.

The uploaded file contained sample business data that would be processed by the pipeline. The successful upload of the CSV file confirmed that Azure Blob Storage was ready to serve as the source location for the data integration process.

Screenshot_04_Source_Container_CSV_Uploaded

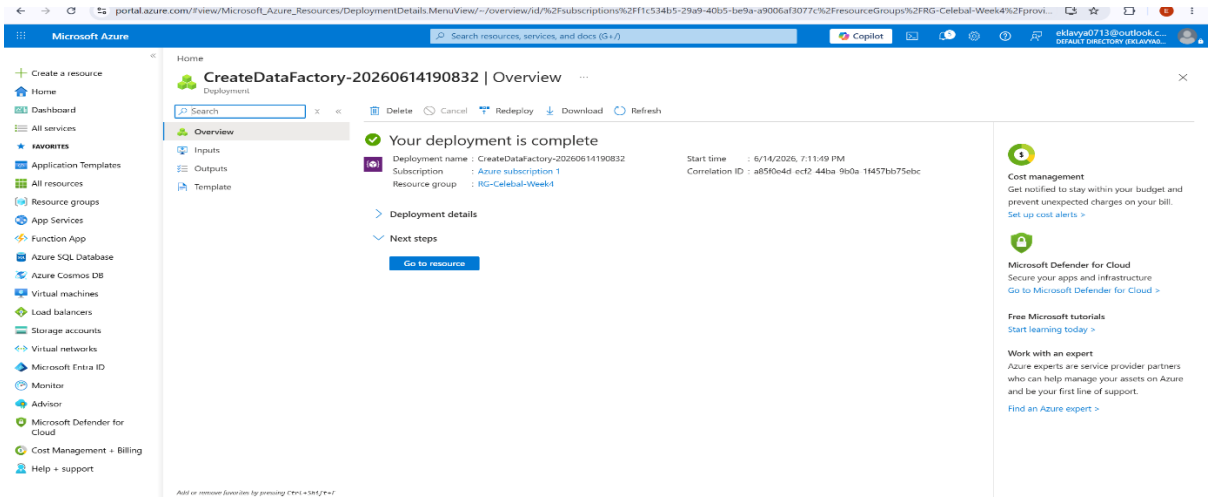


8. Azure Data Factory Creation

An Azure Data Factory instance named **ADF-Eklavya-Week4** was created to build and manage the data pipeline.

Azure Data Factory is a fully managed cloud service used for orchestrating and automating data movement and transformation processes. It provides an intuitive graphical interface that allows users to design data workflows without extensive coding.

Screenshot_05_ADF_Created



9. Exploring Azure Data Factory Studio

After deployment, Azure Data Factory Studio was launched. The Studio environment provides three major sections:

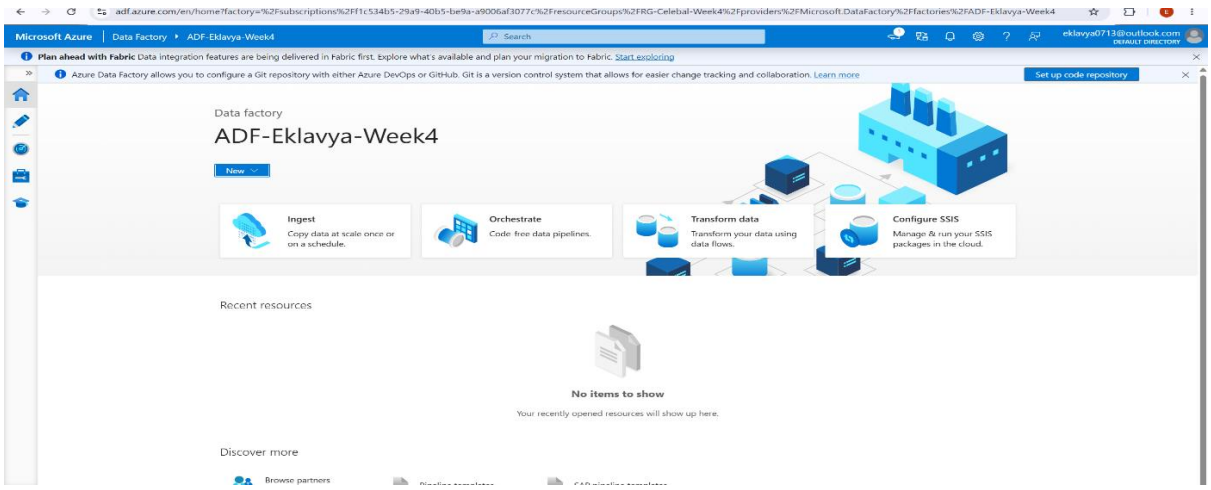
Author: Used to create pipelines, datasets, data flows, and triggers.

Monitor: Used to monitor pipeline runs, execution history, and activity status.

Manage: Used to configure Linked Services, Integration Runtimes, and security settings.

These sections provide all the tools required to design, execute, and manage data integration workflows.

Screenshot_06_ADF_Home_Page



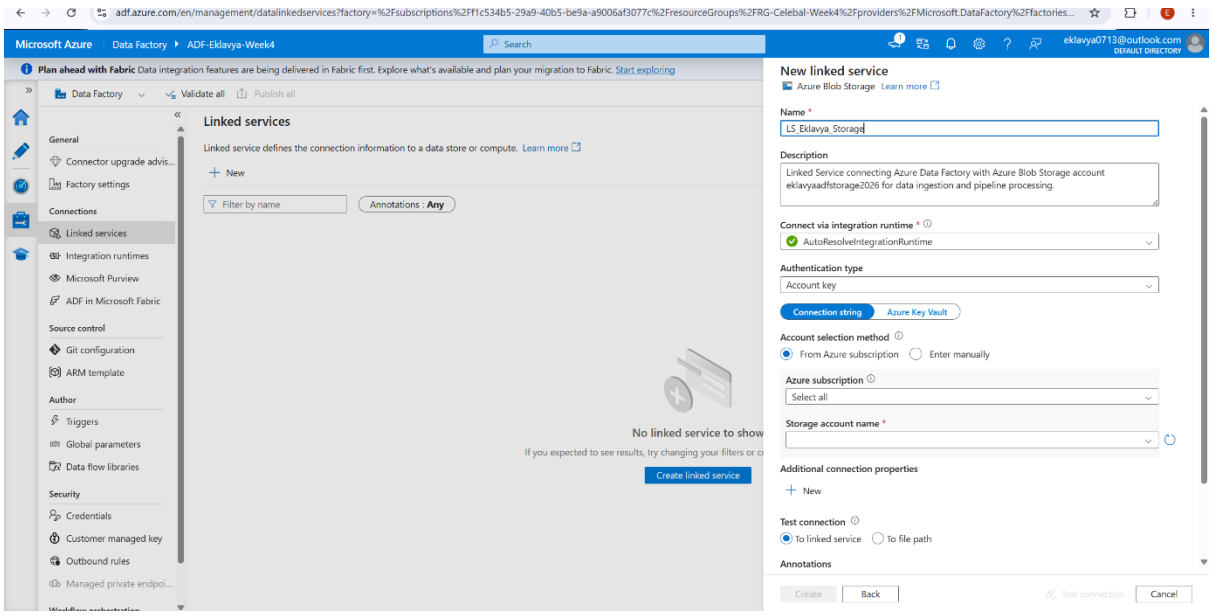
10. Linked Service Configuration

A Linked Service named **LS_Eklavya_Storage** was created to establish connectivity between Azure Data Factory and Azure Blob Storage.

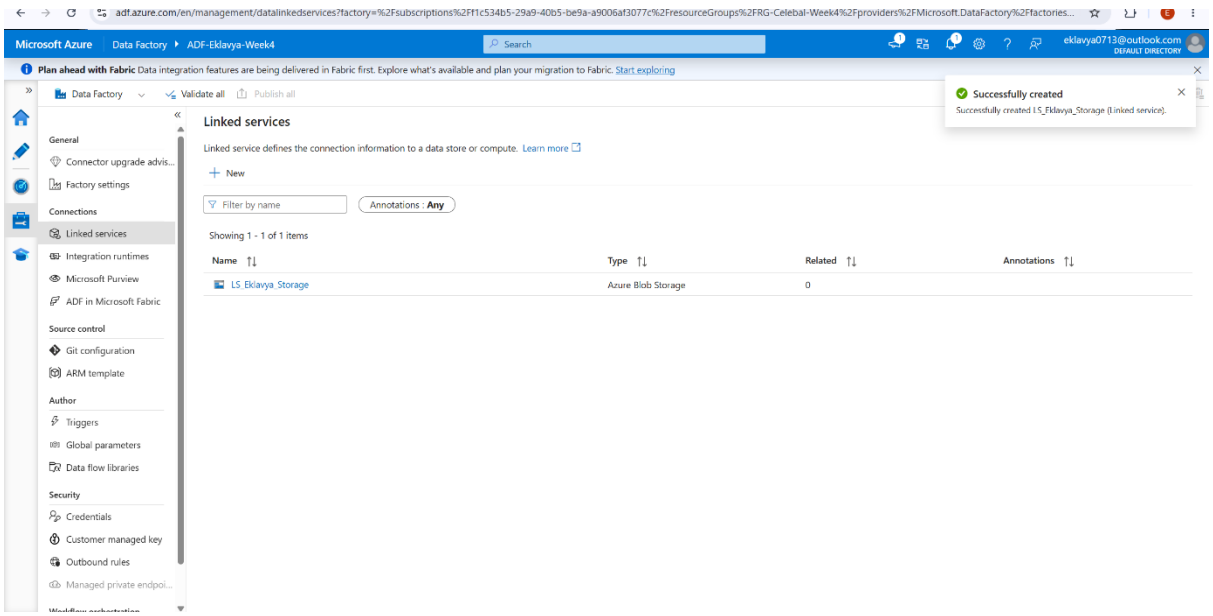
A Linked Service acts as a connection object that stores authentication details and connection information required for Azure Data Factory to access external resources.

After configuring the Storage Account connection, the connection was tested successfully and the Linked Service was created.

Screenshot_07_Linked_Service_Creation



Screenshot_08_Linked_Service_Created



11. Dataset Creation

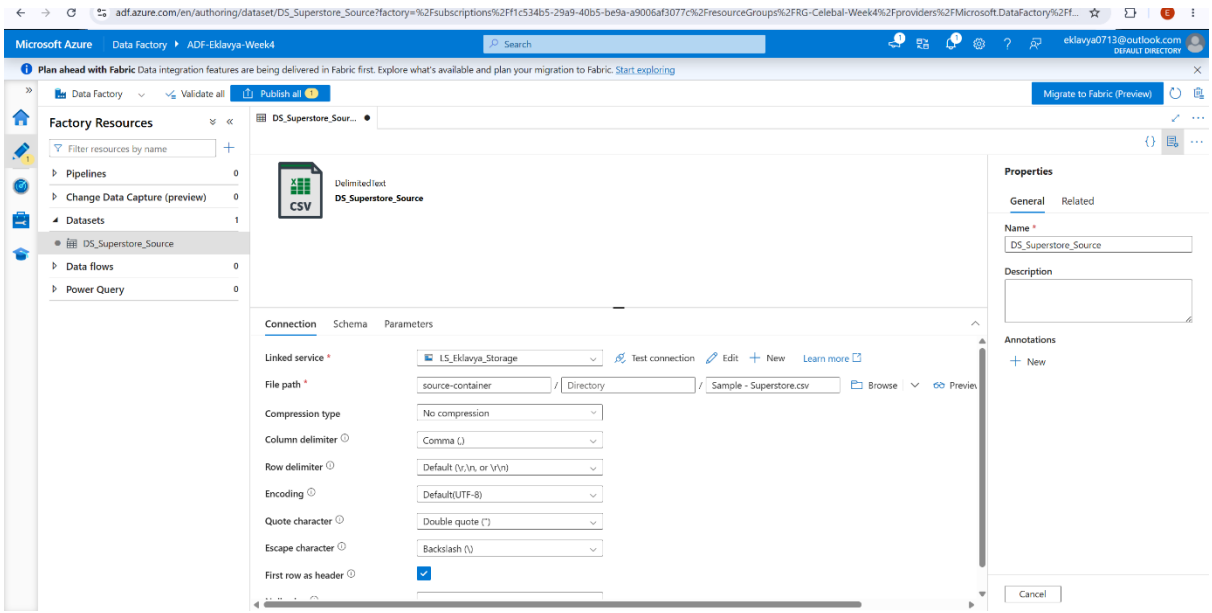
Datasets were created to represent the source and destination files used by the pipeline.

The source dataset, named **DS_Superstore_Source**, was configured to access the Sample-Superstore.csv file stored in the source-container.

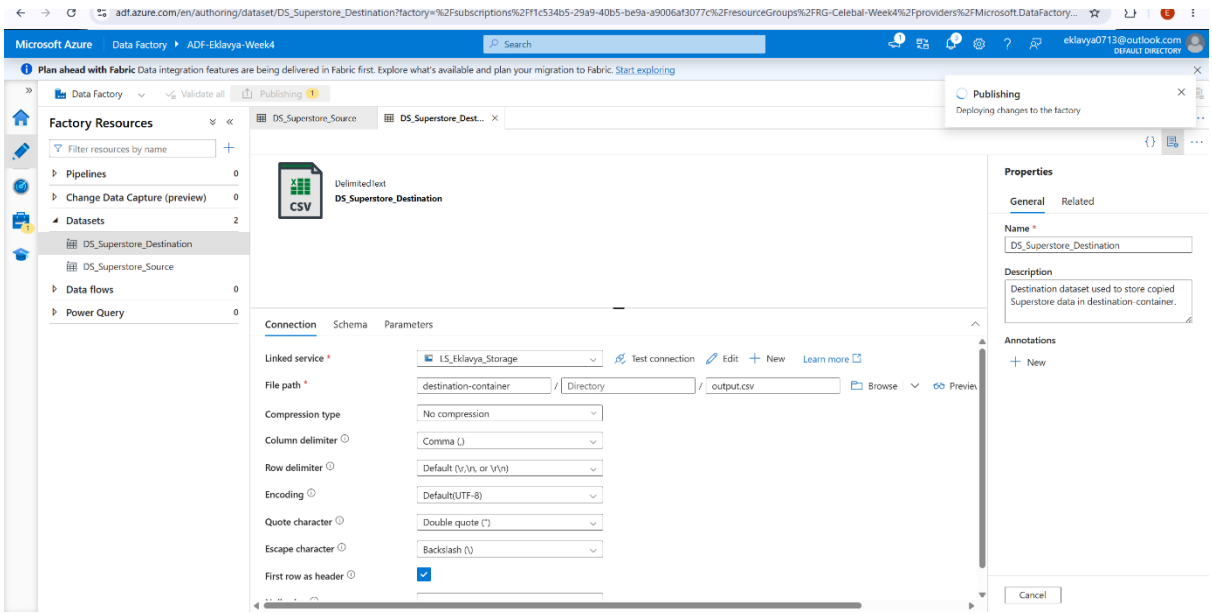
The destination dataset, named **DS_Superstore_Destination**, was configured to store the output file in the destination-container.

Datasets provide Azure Data Factory with information about the structure and location of the data involved in pipeline operations.

Screenshot_09_Source_Dataset



Screenshot_10_Destination_Dataset



12. Metadata Validation Using Get Metadata Activity

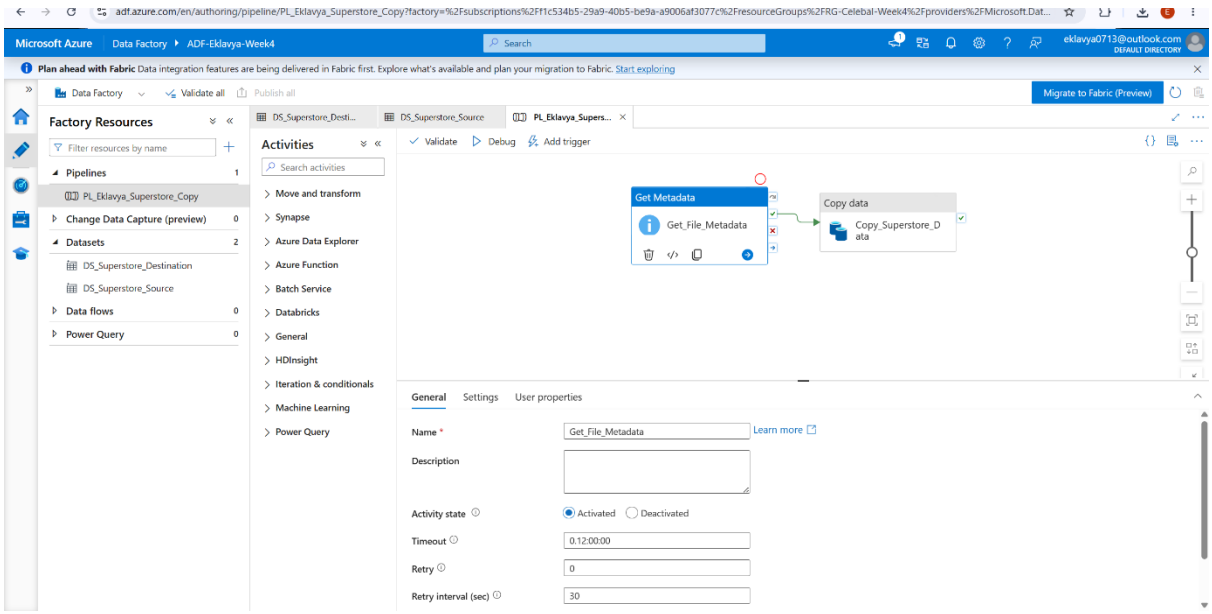
A Get Metadata activity was added to the pipeline to validate the source file before performing the data transfer operation.

The activity was configured to retrieve important metadata properties including:

- File existence
- File size
- Last modified date and time

Metadata validation ensures that the source file is available and accessible before data movement begins, thereby reducing the possibility of pipeline failures.

Screenshot_11_Get_Metadata_Activity



---

13. Pipeline Development

A pipeline named **PL_Eklavya_Superstore_Copy** was created to implement the end-to-end data movement process.

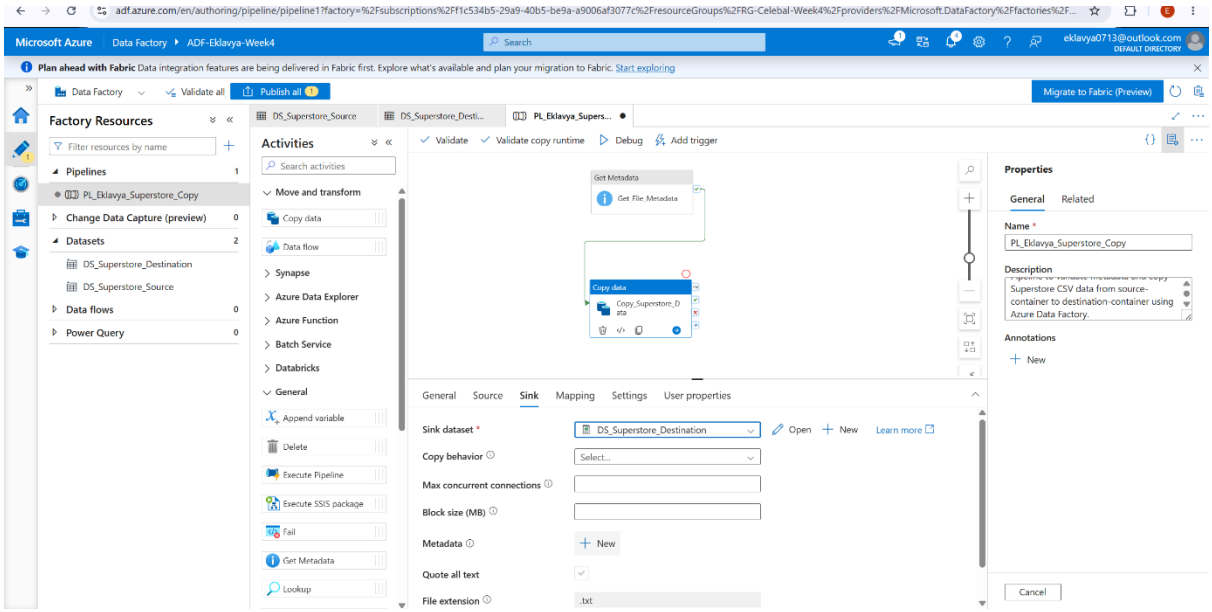
The pipeline consisted of two activities:

1. Get Metadata Activity
2. Copy Data Activity

The Get Metadata activity was executed first to validate the source file. After successful validation, the Copy Data activity transferred the CSV file from the source-container to the destination-container.

The activities were connected sequentially to ensure a smooth and reliable workflow.

Screenshot_12_Pipeline_Design_GetMetadata_CopyData



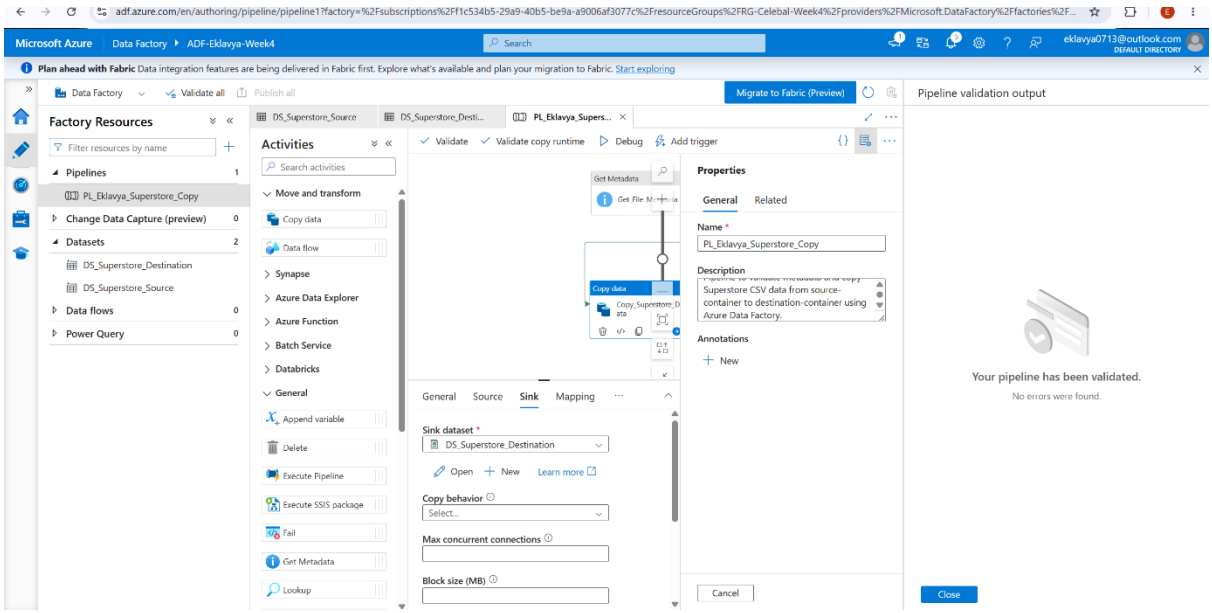
---

14. Pipeline Validation

Before executing the pipeline, a validation check was performed to verify that all configurations were correct.

The validation process confirmed that the pipeline contained no errors and was ready for execution.

Screenshot_13_Pipeline_Validation_Success



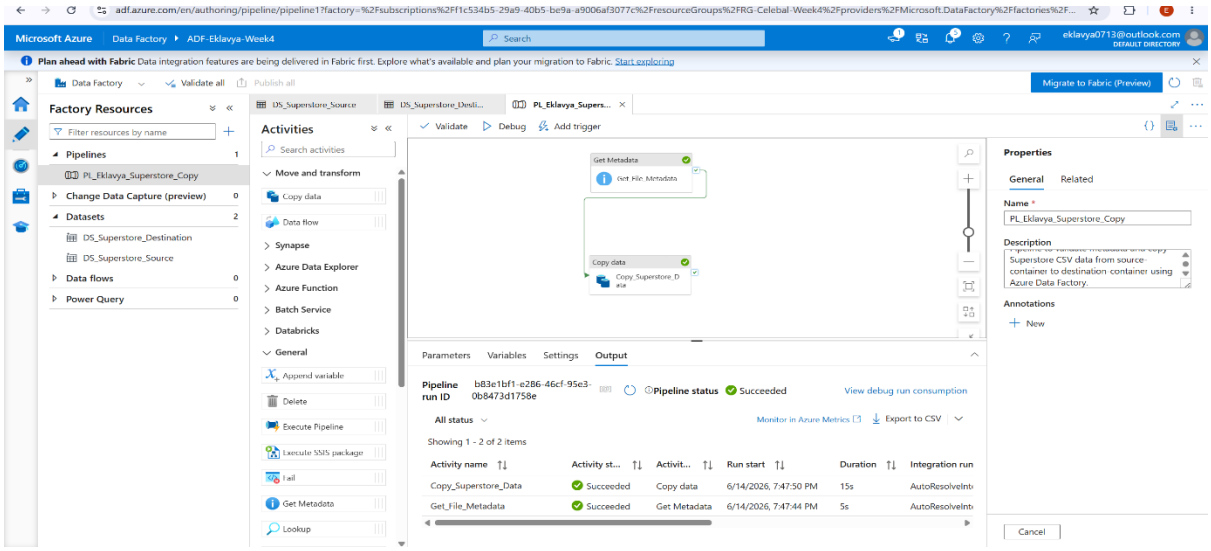
15. Pipeline Execution

The pipeline was executed using the Debug option available in Azure Data Factory.

During execution, the Get Metadata activity successfully retrieved the required file information, and the Copy Data activity completed the data transfer operation.

The successful execution demonstrated that the pipeline configuration was correct and that Azure Data Factory could communicate properly with Azure Blob Storage.

Screenshot_14_Pipeline_Debug_Run_Success

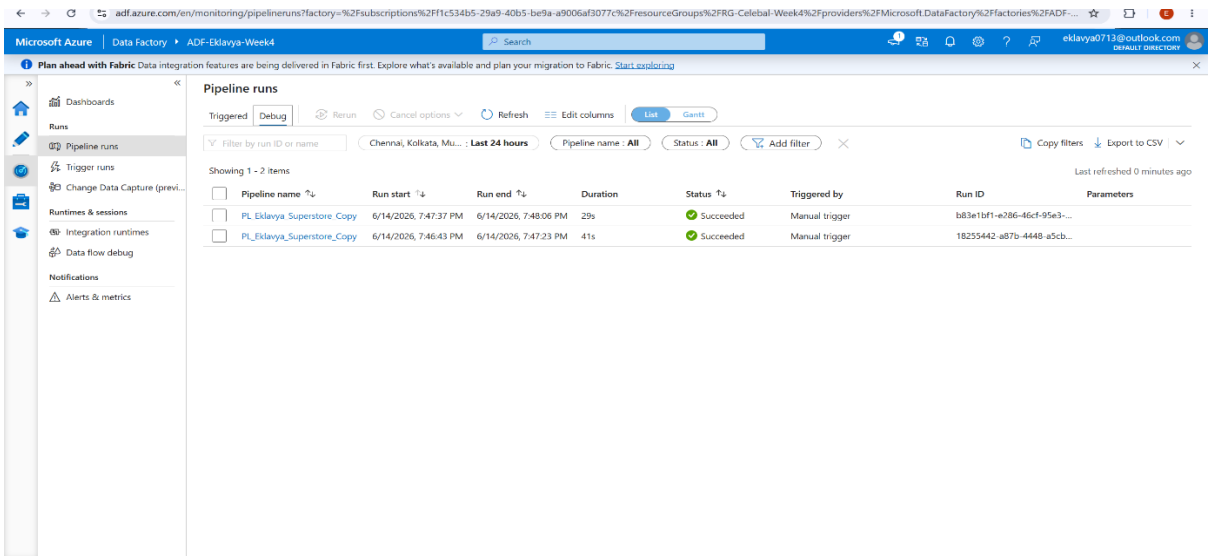


16. Monitoring Pipeline Execution

The Monitor section of Azure Data Factory was used to track the execution status of the pipeline.

The monitoring dashboard displayed execution details, activity status, trigger information, and overall pipeline performance. The final execution status was recorded as **Succeeded**, indicating successful completion of all pipeline activities.

Screenshot_15_Debug_Execution_Success

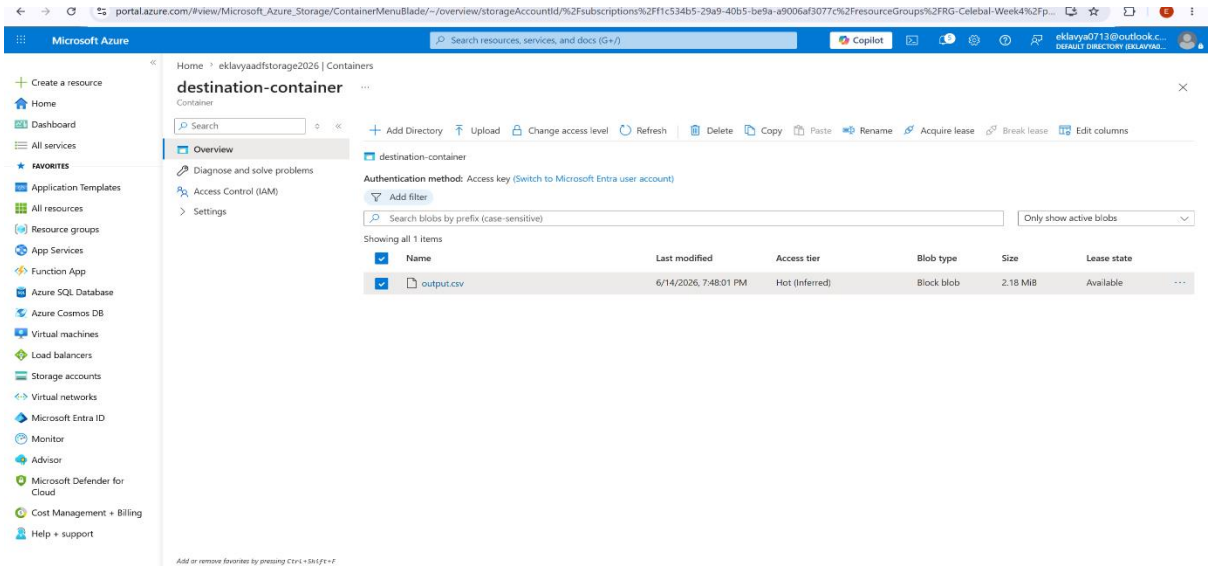


17. Output Verification

After successful pipeline execution, the output file named **output.csv** was generated inside the destination-container.

The presence of the output file confirmed that the data had been copied successfully from the source location to the destination location. This served as final verification that the end-to-end pipeline functioned correctly.

Screenshot_16_Destination_Container_Output_File



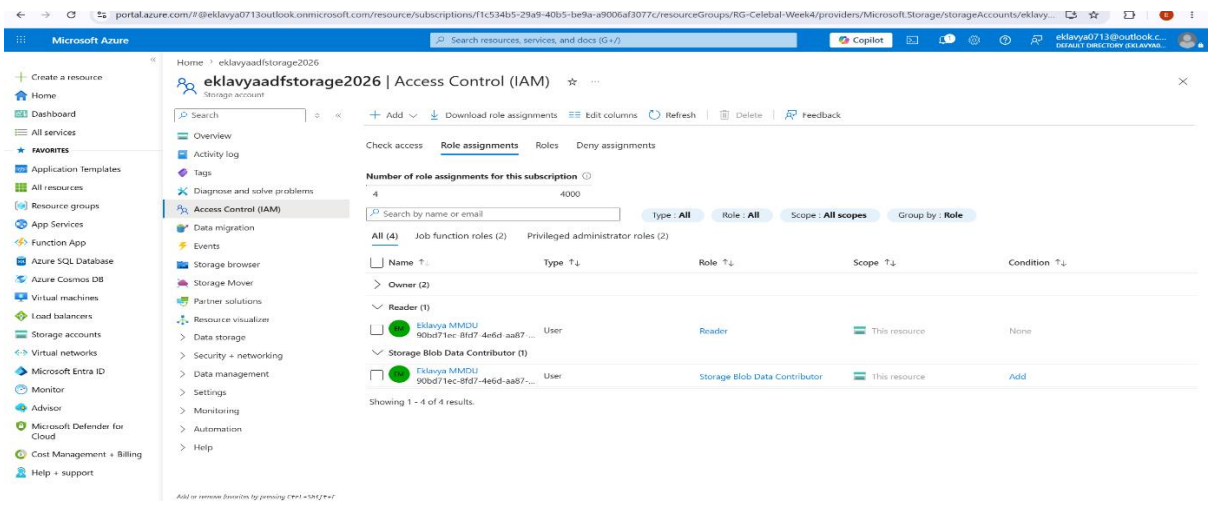
18. IAM Role Assignment

To ensure secure access and proper permissions, Azure IAM roles were assigned.

The Reader role was assigned to provide view access to Azure resources. The Storage Blob Data Contributor role was assigned to allow access to Blob Storage operations required by the data pipeline.

These role assignments helped establish secure communication between Azure Data Factory and Azure Storage resources.

Screenshot_17_IAM_Role_Assignments



19. Result

The Azure Data Factory pipeline executed successfully and achieved all project objectives.

The CSV file stored in Azure Blob Storage was validated through the Get Metadata activity and copied successfully to the destination container using the Copy Data activity. Pipeline execution was monitored successfully, and the output file was generated without any errors.

The project demonstrated successful integration between Azure Blob Storage and Azure Data Factory.

---

20. Conclusion

This assignment provided practical exposure to Microsoft Azure cloud services and data engineering concepts. Through this implementation, a complete end-to-end data pipeline was successfully developed using Azure Blob Storage and Azure Data Factory.

The project helped in understanding cloud resource management, storage services, Linked Services, datasets, metadata validation, pipeline development, execution monitoring, and access control using IAM roles. The successful completion of the pipeline demonstrated the effectiveness of Azure Data Factory as a cloud-based data integration platform and strengthened practical knowledge of modern cloud data engineering workflows.